

Documentación de Excepciones del Sistema de Gestión de Bóvedas

Esta sección documenta la jerarquía de excepciones personalizadas diseñada para el sistema de transporte de valores, basada en el flujo de trabajo y las clases fundamentales identificadas. Estas excepciones son cruciales para garantizar la robustez, claridad y facilidad de depuración del sistema en un entorno de misión crítica.

Todas las excepciones personalizadas heredan de una clase base común, `BovedaException`, que a su vez deriva de `std::runtime_error`, integrándose así con el sistema de manejo de excepciones estándar de C++.

Jerarquía de Excepciones

La siguiente estructura muestra la relación de herencia de las clases de excepción (utilizando caracteres ASCII para compatibilidad con `verbatim`):

```
std::exception
  L-- std::runtime_error
      L-- BovedaException
          |-- SaldoInsuficienteException
          |-- ActivoNoDisponibleException
          |-- OperacionInvalidaException
          |-- TipoOperacionNoSoportadoException
          |-- DatosInvalidosException
          |-- BovedaNoEncontradaException
          |-- EntidadBancariaNoEncontradaException
          |-- TransportadoraNoDisponibleException
          |-- ConfiguracionInvalidaException
          L-- ErrorInternoSistemaException
```

Clases de Excepción

```
class BovedaException : public std::runtime_error
```

```
class BovedaException : public std::runtime_error {
public:
    explicit BovedaException(const std::string& message);
};
```

Propósito: Esta es la clase base para **todas** las excepciones personalizadas específicas del dominio del sistema de gestión de bóvedas. Proporciona una interfaz común y permite la captura genérica de cualquier error relacionado con la lógica de negocio de nuestro sistema.

Hereda de: `std::runtime_error`, lo que la hace compatible con el manejo de excepciones estándar de C++ para errores en tiempo de ejecución.

Constructor:

- `explicit BovedaException(const std::string& message)`
 - **Parámetro message:** Un mensaje descriptivo del error que ocurrió. Este mensaje se puede recuperar con el método `what()`.

Uso Típico: No se lanza directamente. Su función principal es servir como base para otras excepciones más específicas y permitir la captura polimórfica de errores de la bóveda.

Excepciones Relacionadas con el Estado y los Fondos

```
class SaldoInsuficienteException : public BovedaException {
public:
    explicit SaldoInsuficienteException(const std::string& message = "Error de
fondos: Saldo insuficiente en la boveda de origen.");
};
```

Propósito: Indica que una operación no puede completarse debido a que la bóveda de origen (o la entidad bancaria) no posee la cantidad de fondos requerida.

Escenario Típico:

- Un intento de enviar S/ 1 millón con solo S/ 800,000 disponibles.

```
class ActivoNoDisponibleException : public BovedaException {
public:
    explicit ActivoNoDisponibleException(const std::string& message = "Error de
activo: El tipo de activo solicitado no esta disponible o es insuficiente.");
};
```

Propósito: Se lanza si se intenta realizar una operación con un tipo de activo (moneda, bonos, joyas) que la bóveda de origen no posee, o no en la cantidad necesaria, incluso si el saldo total de la bóveda fuera suficiente con otros activos.

Escenario Típico:

- Intentar retirar 1,000 USD de una bóveda que solo almacena Soles Peruanos (PEN).

Excepciones Relacionadas con la Validación de Datos y la Lógica de Negocio

```
class OperacionInvalidaException : public BovedaException {
public:
    explicit OperacionInvalidaException(const std::string& message = "Error de
operacion: La operacion es invalida debido a reglas de negocio o estado.");
};
```

Propósito: Indica una violación de las reglas de negocio o una transición de estado ilegal para una operación. Es una excepción general para errores de lógica de negocio que no encajan en otras categorías más específicas.

Escenarios Típicos:

- Intentar cancelar una operación que ya ha sido marcada como Completada.º Cancelada".
- Intentar crear una operación PROVISION_BCRP (recibir fondos del BCRP) desde una bóveda comercial, cuando esta operación solo es válida para el BCRP.
- Intentar una transferencia interna con un monto que excede un límite predefinido sin la aprobación adecuada.

```
class TipoOperacionNoSoportadoException : public BovedaException {
class TipoOperacionNoSoportadoException : public BovedaException {
public:
    explicit TipoOperacionNoSoportadoException(const std::string& message = "Error
    de tipo: El tipo de operacion especificado no es soportado.");
};
```

Propósito: Se lanza si el sistema recibe o intenta procesar un tipo de operación que no está definido o no es reconocido por el sistema (ej. un valor de enumeración inválido para `TipoOperacion`).

Escenario Típico:

- Una integración externa intenta crear una transacción con un `TipoOperacion` como `^JUSTE_IN-VENTARIO` que no existe en el catálogo de operaciones del sistema.

```
class DatosInvalidosException : public BovedaException {
class DatosInvalidosException : public BovedaException {
public:
    explicit DatosInvalidosException(const std::string& message = "Error de datos:
    Los datos de entrada son invalidos o tienen formato incorrecto.");
};
```

Propósito: Indica que los datos proporcionados para una operación o entidad son inválidos, tienen un formato incorrecto o no cumplen con restricciones básicas de entrada.

Escenario Típico:

- Intentar crear una operación con un monto negativo o cero.
- Proporcionar un ID de bóveda no numérico o una fecha en un formato incorrecto.

Excepciones Relacionadas con la Disponibilidad y Existencia de Entidades

```
class BovedaNoEncontradaException : public BovedaException {
class BovedaNoEncontradaException : public BovedaException {
public:
    explicit BovedaNoEncontradaException(const std::string& message = "Error de
    entidad: La boveda especificada no fue encontrada.");
};
```

Propósito: Se lanza cuando el sistema intenta interactuar con una bóveda (ya sea de origen o destino) cuyo identificador no corresponde a ninguna bóveda registrada en el sistema.

Escenario Típico:

- Ingresar un ID de bóveda (`BVD_XYZ`) incorrecto o inexistente al programar una transferencia.

```
class EntidadBancariaNoEncontradaException : public BovedaException {
class EntidadBancariaNoEncontradaException : public BovedaException {
public:
    explicit EntidadBancariaNoEncontradaException(const std::string& message = "
    Error de entidad: La entidad bancaria especificada no fue encontrada.");
};
```

Propósito: Indica que una operación hace referencia a una entidad bancaria (un banco o el BCRP) que no está registrada o no existe en el sistema.

Escenario Típico:

- Intentar una TRASLADO_INTERBANCARIO a un banco con un RUC que no corresponde a ninguna entidad bancaria conocida.

```
class TransportadoraNoDisponibleException : public BovedaException {
public:
    explicit TransportadoraNoDisponibleException(const std::string& message = "
        Error de servicio: La transportadora no esta disponible o no cubre la zona.");
};
```

Propósito: Se lanza si la empresa transportadora de valores asignada a una operación no está disponible para el servicio, no cubre la ubicación de destino, o no está habilitada en el sistema.

Escenario Típico:

- Intentar programar un envío con "Transportes XYZ.^a una ubicación remota que no está dentro de su área de servicio.
- La transportadora seleccionada está marcada como inactiva.^{en} el sistema.

Excepciones de Sistema / Internas

```
class ConfiguracionInvalidaException : public BovedaException {
public:
    explicit ConfiguracionInvalidaException(const std::string& message = "Error de
        configuracion: El sistema no esta configurado correctamente.");
};
```

Propósito: Indica un problema con la configuración interna del sistema que impide el correcto funcionamiento. Estos errores suelen ser detectados al inicio o durante la inicialización de módulos.

Escenario Típico:

- Los identificadores predefinidos para el BCRP o ciertas bóvedas clave no han sido cargados o son incorrectos en la configuración.

```
class ErrorInternoSistemaException : public BovedaException {
public:
    explicit ErrorInternoSistemaException(const std::string& message = "Error
        interno: Se ha producido un fallo inesperado en el sistema.");
};
```

Propósito: Una excepción de "último recurso" para errores inesperados y no clasificados que indican un fallo en la lógica interna del sistema o una condición que no debería ocurrir bajo operación normal.

Escenario Típico:

- Un fallo en una dependencia interna, una corrupción de datos en memoria o un estado de objeto inconsistente que no puede ser atribuido a una excepción más específica.

Principios de Uso de las Excepciones

Lanzar (throw) Cuando se detecta una condición de error que la función actual no puede resolver, se crea una instancia de la excepción más específica y se lanza utilizando la palabra clave **throw**.

```
throw SaldoInsuficienteException("La cuenta de origen no tiene fondos suficientes.");
```

Capturar (catch) Se utilizan bloques **try-catch** para interceptar las excepciones lanzadas. Es crucial colocar los bloques **catch** de las excepciones más específicas antes que los más genéricos en una secuencia **try-catch**.

```
try {  
    // Código que puede lanzar una excepción  
    realizarOperacion();  
} catch (const SaldoInsuficienteException& e) {  
    // Tratamiento específico para saldo insuficiente  
    std::cerr << "ERROR DE SALDO: " << e.what() << std::endl;  
} catch (const BovedaException& e) {  
    // Tratamiento genérico para cualquier error del sistema de bóveda  
    std::cerr << "ERROR DEL SISTEMA BOVEDA: " << e.what() << std::endl;  
} catch (const std::exception& e) {  
    // Tratamiento para cualquier otra excepción estándar  
    std::cerr << "ERROR INESPERADO: " << e.what() << std::endl;  
}
```

Mensajes Descriptivos (what()) Todas las clases de excepción heredan el método **what()** de **std::runtime_error** (o **std::exception**). Este método devuelve la cadena de caracteres (el mensaje) que se pasó al constructor de la excepción, proporcionando información vital sobre la causa del error.

Relanzamiento (throw;) En algunos casos, una función puede capturar una excepción, realizar un procesamiento parcial (ej. loguear el error, limpiar recursos locales) y luego relanzarla (**throw;**) para que un nivel superior de la aplicación pueda manejarla de forma más global.

```
try {  
    // Código que lanza excepción  
    throw SaldoInsuficienteException("Faltan fondos.");  
} catch (const BovedaException& e) {  
    // Primer tratamiento: loguear el error  
    std::cerr << "LOG: Error capturado internamente: " << e.what() << std::endl;  
    // Relanzar para tratamiento superior  
    throw;  
}
```